Project Report — Smart MCQ Solver Challenge

1. Abstract

The Smart MCQ Solver Challenge focuses on building an AI system that can answer multiple-choice questions by selecting the most likely correct answers from five options (A–E). Unlike a standard classification task, the competition requires the model to rank the top three answer choices, and submissions are evaluated using the **Mean Average Precision at 3 (MAP@3)** metric.

In this project, three different approaches were explored. The first was a Transformer encoder implemented from scratch using PyTorch to understand the fundamentals of transformer architectures. The second approach involved fine-tuning **DistilBERT**, while the third used **Microsoft DeBERTa-v3-base**, both through the Hugging Face Transformers library. The performance of the fine-tuned models was monitored using Weights & Biases (wandb).

Among all the models, DeBERTa-v3-base achieved the best performance, followed by DistilBERT. The custom Transformer model served as a useful baseline but did not perform as well as the pretrained models. Overall, this project demonstrated the effectiveness of transfer learning for multiple-choice question answering and provided valuable experience with transformer models, model fine-tuning, and experiment tracking.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge is a Kaggle competition that focuses on solving multiple-choice questions using machine learning. Each question contains a prompt and five answer options labeled **A, B, C, D, and E**. The training dataset provides the correct answer for each question, while the test dataset contains only the questions and answer choices. The objective is to predict the top three most likely answers for each question. Model performance is evaluated using the **MAP@3** metric, which gives higher scores when the correct answer is ranked closer to the first position.

2.2 Project Objective

The main objective of this project was to develop and compare different transformer-based models for the multiple-choice question answering task. A Transformer encoder was first implemented from scratch to understand its working principles. After establishing this baseline, pretrained models such as **DistilBERT** and **DeBERTa-v3-base** were fine-tuned to improve prediction accuracy. Another important objective was to gain practical experience with the Hugging Face ecosystem, PyTorch, tokenization techniques, and experiment tracking using Weights & Biases.

2.3 Report Structure

This report begins with an overview of the dataset and the preprocessing steps performed before training. It then explains the tokenization strategy used for the transformer models. The next section describes the implementation of the three models and the training process. Their performance is then compared using the competition evaluation metric and training results. Finally, the report concludes with the key learnings from the project, the challenges faced, and possible improvements for future work.

3. Dataset & Preprocessing

3.1 Dataset Description

The dataset used in this project was provided as part of the **Smart MCQ Solver Challenge** on Kaggle. It consists of multiple-choice questions, where each question contains a prompt and five answer options labeled **A, B, C, D, and E**. The training dataset includes the correct answer for each question, while the test dataset contains only the questions and answer options. The dataset is provided in three CSV files: **train.csv**, **test.csv**, and **sample_submission.csv**. The objective is to predict the top three most likely answer choices for each question, and the model performance is evaluated using the **Mean Average Precision at 3 (MAP@3)** metric.

3.2 Exploratory Data Analysis (EDA)

An exploratory data analysis (EDA) was performed to understand the dataset before model training. The training dataset contains **2,000 multiple-choice questions**, while the test dataset contains **500 questions**. Each record includes a question prompt and five answer options (A–E). The training dataset also contains the correct answer label, whereas the test dataset does not.

The dataset was found to be clean, with **no missing values or mismatched records** in any column. The training data contains **1,758 unique question prompts**, indicating that most questions are unique. Similarly, the answer option columns (A–E) contain more than **300 unique responses** each, providing a diverse set of answer choices.

The target variable consists of **five classes (A, B, C, D, and E)**. The class distribution is fairly balanced, with **option B accounting for approximately 25% of the answers** and **option C around 23%**, while the remaining classes share the rest of the dataset. This balanced distribution helps reduce bias during model training.

Overall, the EDA showed that the dataset is well-structured, free from missing values, and suitable for training transformer-based models for the multiple-choice question answering task.

3.3 Data Preprocessing

Before training the models, the dataset was prepared through a few preprocessing steps. The **train.csv** and **test.csv** files were loaded using Pandas, and the answer labels (**A, B, C, D, and E**) in the training dataset were converted into numerical values for model training. Since the dataset did not contain any missing or mismatched values, no additional data cleaning was required.

For each question, the prompt was paired with all five answer options to create the input format required for multiple-choice transformer models. The text was then tokenized using the Hugging Face tokenizer with a maximum sequence length of **256 tokens**. Padding and truncation were applied to ensure that all input sequences had the same length. Finally, the processed data was converted into the Hugging Face Dataset format, making it ready for efficient training and evaluation.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

For this project, the tokenizer provided with each pretrained model was used through the Hugging Face **AutoTokenizer** class. The best-performing model, **Microsoft DeBERTa-v3-base**, uses a **SentencePiece-based subword tokenizer**, which converts text into smaller meaningful units before passing it to the model. DistilBERT was also trained using its corresponding pretrained tokenizer.

4.2 Justification

Using the pretrained tokenizer ensures that the input text is processed in the same way as during the model's original pretraining. This helps the model better understand the relationship between the question and its answer choices, resulting in improved prediction accuracy. Subword tokenization also allows the model to handle unknown or rare words effectively.

4.3 Implementation Details

The tokenization process was implemented using the Hugging Face Transformers library. For each question, the prompt was paired with every answer option to create five input sequences. A maximum sequence length of **256 tokens** was used, with **padding** and **truncation** enabled to maintain a fixed input size. Batch tokenization was used to improve preprocessing speed before training.

4.4 Experimental Comparison

Both **DistilBERT** and **DeBERTa-v3-base** tokenizers were evaluated during the project. While DistilBERT offered faster preprocessing and lower computational cost, DeBERTa's tokenizer produced better input representations for the task, resulting in higher validation performance. Therefore, the DeBERTa tokenizer was selected for the final model.

5. Modelling and Experimentation

5.1 Model 1: DeBERTa-v3-base (Fine-Tuned Transformer)

The main model used in this project was **Microsoft DeBERTa-v3-base**, a pretrained transformer model known for its strong language understanding capabilities. It improves upon earlier transformer models by using a disentangled attention mechanism, which helps the model better understand both the meaning of words and their positions in a sentence. This makes it well suited for multiple-choice question answering, where understanding the relationship between the question and each answer option is important.

The model was implemented using Hugging Face's **AutoModelForMultipleChoice**, which automatically scores all five answer options and predicts the most likely correct one. During training, the entire model was fine-tuned without freezing any layers so that it could adapt fully to the competition dataset. The model was trained for **15 epochs** using the **Adafactor** optimizer with a learning rate of $\mathbf{2 \times 10^{-5}}$, weight decay of **0.02**, and a batch size of **8**. No quantization techniques were applied during training.

5.2 Model 2: DistilBERT (Fine-Tuned Transformer)

The second model used in this project was **DistilBERT**, which is a smaller and faster version of BERT. Although it contains fewer parameters, it still performs well on many natural language processing tasks while requiring less memory and training time.

DistilBERT was trained using the same multiple-choice classification approach as DeBERTa. The model was fine-tuned for **5 epochs** using the **AdamW** optimizer with a learning rate of $\mathbf{2 \times 10^{-5}}$ and weight decay of **0.02**. The same preprocessing and tokenization pipeline was used to ensure a fair comparison between the two models.

This model served as a lightweight baseline and helped compare the trade-off between training speed and prediction accuracy. The best model checkpoint was selected based on the highest validation MAP@3 score.

5.3 Model 3: Transformer Encoder Built from Scratch

To better understand how transformer models work internally, a simple Transformer encoder was implemented from scratch using PyTorch. Unlike the previous two models, this model did not use any pretrained weights. Instead, all model parameters were initialized randomly and learned only from the competition dataset.

The architecture consisted of an embedding layer, positional embeddings, four Transformer encoder layers, mean pooling, and a final linear layer for predicting the correct answer. The model was trained for **5 epochs** using the **AdamW** optimizer with a learning rate of $\mathbf{3 \times 10^{-4}}$ and weight decay of **0.01**.

Although this model achieved a low training loss, its performance was lower than the pretrained transformer models because it had to learn language patterns from a relatively small training dataset without the benefit of large-scale pretraining.

6. Performance and Comparative Analysis

6.1 Evaluation Metrics

The performance of all models was evaluated using **Mean Average Precision at 3 (MAP@3)**, which is the official evaluation metric of the Kaggle competition. This metric rewards models that rank the correct answer higher in their top three predictions. In addition to MAP@3, validation loss was monitored during training to evaluate how well each model generalized to unseen data. Training loss was also tracked to monitor the learning progress of the models.

6.2 Training Performance

The training results showed clear differences between the three models. **DistilBERT** learned quickly and achieved a validation MAP@3 of approximately **0.96** after only **5 epochs**, while its validation loss steadily decreased throughout training.

DeBERTa-v3-base required a longer training period but produced the best overall results. After **15 epochs**, it achieved a validation MAP@3 of approximately **0.99**, with a much lower validation loss than the other models. Training progress for both DeBERTa and DistilBERT was monitored using **Weights & Biases (wandb)**.

The Transformer model built from scratch showed a steady decrease in training loss over five epochs. However, since it was trained without pretrained knowledge and was not evaluated using the same validation pipeline, its overall performance was lower than the fine-tuned transformer models.

6.3 Comparative Analysis

The comparison of all three models shows that pretrained transformer models perform significantly better than a transformer trained entirely from scratch. **DeBERTa-v3-base** achieved the highest prediction accuracy but required the longest training time and the most computational resources. **DistilBERT** offered a good balance between performance and efficiency, achieving competitive results while training much faster. The custom Transformer model provided useful insights into transformer architecture but was limited by the relatively small training dataset.

Model	Validation Loss	Validation MAP@3	Training Cost
Transformer (From Scratch)	0.70	0.75	Low

DistilBERT	0.746	0.956	Medium
DeBERTa-v3-base	0.221	0.989	High

6.4 Kaggle Performance

For the final competition submission, the **DistilBERT** model was used to generate predictions. The model ranked the top three most probable answer options for each question, and the predictions were saved in the required **submission.csv** format. The final submission achieved a **Kaggle leaderboard score of 0.75062**, demonstrating that the fine-tuned transformer model was able to perform well on unseen multiple-choice questions.

7. Conclusion

This project provided valuable hands-on experience in building an AI-based system for solving multiple-choice questions using transformer models. Throughout the project, I learned how to preprocess textual data, apply tokenization, fine-tune pretrained language models such as DeBERTa and DistilBERT, and evaluate their performance using the MAP@3 metric. Among the models tested, DeBERTa achieved better performance due to its stronger contextual understanding. One of the main challenges was selecting suitable hyperparameters and improving model performance through experimentation. In the future, the project can be enhanced by exploring larger language models, ensemble techniques, Retrieval-Augmented Generation (RAG), and more advanced hyperparameter tuning methods to achieve better prediction accuracy.